

Circuit Documentation

Circuit Summary

This circuit is an Arduino-based ultrasonic scanning system. An Arduino UNO controls a servo motor while measuring distance with an HC-SR04 ultrasonic sensor.

The servo sweeps from 0° to 180° and then back to 0° in 1° increments. At each angle, the HC-SR04 sensor measures the distance to an object. The Arduino sends the measured angle and distance to the serial monitor at 9600 baud.

The reported data format is:

```
angle,distance.
```

For example:

```
45,32.
```

where the angle is in degrees and the distance is in centimeters.

Power note: The servo and ultrasonic sensor are both powered from the Arduino 5 V pin. A servo can draw significant current, particularly during movement or startup. For reliable operation, use a separate regulated 5 V servo supply if needed, and connect its ground to the Arduino ground.

Component List

Arduino UNO

The Arduino UNO is the circuit's microcontroller and control device. It:

- Generates the trigger signal for the ultrasonic sensor.
- Measures the ultrasonic echo pulse duration.
- Controls the servo position using a PWM-capable output.
- Calculates distance in centimeters.
- Sends scan results through its serial interface.

HC-SR04 Ultrasonic Sensor

The HC-SR04 measures distance by transmitting an ultrasonic pulse and measuring the time required for the echo to return.

Its connections are:

- **VCC:** 5 V power supply.
- **TRIG:** Input used to start a measurement.
- **ECHO:** Output pulse whose duration represents the echo travel time.
- **GND:** Ground reference.

Servo

The servo rotates the ultrasonic sensor or an attached mechanism through the scanning range. Its connections are:

- **VCC:** 5 V power supply.
- **PWM:** Control signal from the Arduino.
- **GND:** Ground reference.

Wiring Details

Arduino UNO

Arduino Pin	Connected To	Function
5V	HC-SR04 VCC	Supplies power to the ultrasonic sensor
5V	Servo VCC	Supplies power to the servo
GND	HC-SR04 GND	Common ground
GND	Servo GND	Common ground
D8	HC-SR04 TRIG	Starts an ultrasonic measurement
D9	HC-SR04 ECHO	Receives the echo timing pulse
D11	Servo PWM	Controls the servo position

The Arduino's D8, D9, and D11 pins are configured by the program as follows:

- **D8:** Digital output for the ultrasonic trigger.
- **D9:** Digital input for the ultrasonic echo.
- **D11:** Servo control output.

HC-SR04 Ultrasonic Sensor

Sensor Pin	Connected To	Function
VCC	Arduino 5V	Sensor power
GND	Arduino GND	Ground reference
TRIG	Arduino D8	Measurement trigger input
ECHO	Arduino D9	Echo timing output

The Arduino initiates a measurement by applying a pulse to **TRIG**. The sensor then drives **ECHO** high for a duration proportional to the distance measured.

Servo

Servo Pin	Connected To	Function
VCC	Arduino 5V	Servo power
GND	Arduino GND	Ground reference
PWM	Arduino D11	Position control signal

The servo is commanded to move between 0° and 180°. The Arduino sends the control signal through the Servo library.

Documented Code

Included Library

```
#include <Servo.h>
```

The Servo library provides the functions required to attach and control the servo motor.

Pin Definitions

```
const int SERVO_PIN = 11;  
const int TRIG_PIN = 8;  
const int ECHO_PIN = 9;
```

These constants identify the Arduino pins used by the circuit:

- SERVO_PIN controls the servo.
- TRIG_PIN starts an HC-SR04 measurement.
- ECHO_PIN receives the echo pulse.

Scan and Measurement Constants

```
const int MIN_ANGLE = 0;  
const int MAX_ANGLE = 180;  
const int ANGLE_STEP = 1;  
const int SWEEP_DELAY = 15;
```

These values define the scanning behavior:

- The minimum servo angle is 0°.
- The maximum servo angle is 180°.
- The servo moves in 1° increments.
- The program waits 15 milliseconds after each servo movement before measuring distance.

```
const float SPEED_COEF = 58.2;
```

The program uses the relationship:

distance in centimeters = echo duration in microseconds / 58.2

The factor accounts for the round-trip travel time of sound.

Servo Object

```
Servo myServo;
```

This creates a servo control object named myServo.

Setup Function

```
void setup() {  
  pinMode(TRIG_PIN, OUTPUT);  
  pinMode(ECHO_PIN, INPUT);  
  myServo.attach(SERVO_PIN);  
  Serial.begin(9600);  
}
```

The setup() function runs once when the Arduino starts.

It performs the following actions:

1. Configures the ultrasonic trigger pin as an output.
2. Configures the ultrasonic echo pin as an input.
3. Attaches the servo control object to Arduino D11.
4. Starts serial communication at 9600 baud.

Main Loop

```
void loop() {  
    sweepAndMeasure(MIN_ANGLE, MAX_ANGLE, ANGLE_STEP);  
  
    sweepAndMeasure(MAX_ANGLE, MIN_ANGLE, -ANGLE_STEP);  
}
```

The main loop continuously performs two scans:

1. A forward scan from 0° to 180°.
2. A reverse scan from 180° back to 0°.

After the reverse scan completes, the process repeats.

Sweep and Measure Function

```
void sweepAndMeasure(int start_angle, int end_angle, int steps){  
    for (int curr_angle = start_angle; (steps > 0) ? (curr_angle <=  
end_angle) : (curr_angle >= end_angle);  
        curr_angle += steps){  
        myServo.write(curr_angle);  
        delay(SWEEP_DELAY);  
        int distance = calculate_distance();  
        printData(curr_angle, distance);  
    }  
}
```

The sweepAndMeasure() function controls one complete servo sweep.

Parameters

- start_angle: The angle at which the sweep begins.
- end_angle: The final angle of the sweep.
- steps: The angle increment. A positive value moves upward in angle, while a negative value moves downward.

Operation

For each angle, the function:

1. Moves the servo to the current angle.
2. Waits 15 milliseconds for the servo to move.
3. Measures the distance using `calculate_distance()`.
4. Prints the angle and distance using `printData()`.

The conditional loop expression supports both forward and reverse sweeps.

Distance Measurement Function

```
int calculate_distance() {
    digitalWrite(TRIG_PIN, LOW);
    delay(2);
    digitalWrite(TRIG_PIN, HIGH);
    delay(10);
    digitalWrite(TRIG_PIN, LOW);

    long rtt = pulseIn(ECHO_PIN, HIGH);

    return static_cast<int>(rtt / SPEED_COEF);
}
```

The `calculate_distance()` function triggers the HC-SR04 and converts the echo duration into centimeters.

Trigger Operation

1. The trigger pin is driven LOW.
2. The program waits briefly.
3. The trigger pin is driven HIGH.
4. The program waits 10 milliseconds.
5. The trigger pin is driven LOW again.

Echo Measurement

```
long rtt = pulseIn(ECHO_PIN, HIGH);
```

The `pulseIn()` function measures how long the echo pin remains HIGH. This duration is the round-trip travel time of the ultrasonic pulse.

Distance Calculation

```
return static_cast<int>(rtt / SPEED_COEF);
```

The measured time is divided by 58.2 to estimate the distance in centimeters. The result is converted to an integer before being returned.

Implementation note: The standard HC-SR04 trigger pulse is typically approximately 10 microseconds, while this code holds the trigger HIGH for 10 milliseconds because it uses `delay(10)`. For best sensor compatibility, the trigger section should normally use `delayMicroseconds(10)` instead.

A typical corrected trigger sequence would be:

```
digitalWrite(TRIG_PIN, LOW);  
delayMicroseconds(2);  
digitalWrite(TRIG_PIN, HIGH);  
delayMicroseconds(10);  
digitalWrite(TRIG_PIN, LOW);
```

The current code also does not specify a timeout for `pulseIn()`. If no echo is received, the function can wait for its default timeout and return zero, which may be reported as a zero-centimeter distance.

Data Output Function

```
void printData(int angle, int dist){  
    Serial.print(angle);  
    Serial.print(",");  
    Serial.print(dist);  
    Serial.print(".");  
}
```

The `printData()` function sends the scan result to the serial monitor. Each result contains:

1. The servo angle.
2. A comma separator.
3. The measured distance.
4. A period terminator.

Example output:

```
0,48.1,47.2,46.
```

The code does not send a newline after each measurement, so all measurements are placed on the same serial line unless the receiving application interprets the period as a record delimiter.

Operating Sequence

1. The Arduino starts and configures the I/O pins.
2. The servo is attached to Arduino D11.
3. Serial communication begins at 9600 baud.
4. The servo moves from 0° to 180°.
5. At every degree:
 - The servo moves.
 - The Arduino waits briefly.
 - The HC-SR04 measures distance.
 - The angle and distance are transmitted.
6. The servo moves from 180° back to 0°.
7. The forward and reverse scanning cycle repeats continuously.